CSE 4304-Data Structures Lab. Winter 2022
Date: August 29, 2022
Target Group: All
Topic: Heap, Heapsort, Priority Queues

Instructions:

- Task naming format: fullID_T01L05_2A.c/CPP
- If you find any issues in problem description/test cases, comment in the google classroom.
- If you find any test case that is tricky that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to easily recall the solution in the future.
- Obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Group	Tasks
1B	1 2 3 5
2B	1 2 5 6
2A	1 2 6 11
1A	1 2 6 11

Task-01: Implementing the basic operations of a **Heap**.

Suppose an arbitrary array of size N is given as input. Your task is to build a **max**-heap from the set of numbers and sort them using Heap-sort.

Take input as long you don't get -1. For each test case, show the state of Max-Heap and the Sorted array.

Input	**Output**
4 1 3 2 16 9 10 14 8 7 -1	Max Heap: 16 14 10 8 7 9 3 2 4 1 Sorted: 16 14 10 9 8 7 4 3 2 1
7 9 6 19 8 17 11 2 5 3 13 -1	Max Heap:19 13 17 9 8 6 11 2 5 3 7 Sorted: 19 17 13 11 9 8 7 6 5 3 2

Note:

- Use Separate functions for 'heapify', 'Build_max_heap', 'Heap_sort'.
- Only call the build_max_heap function once in the heapsort function (at the beginning to create a heap)!!

Task 2

Use the Heap that you created in Task 1 and convert it into a **'Min Priority Queue'** and implement the following functionalities:

1. int **Heap_Minimim**(int heap[]): Returns the minimum value.
2. int **Heap_extract_min**(int heap[]): Removes the minimum value and returns it.
3. **Min_heap_insert**(int value, int heap[]): Inserts the 'value' into the heap and makes necessary arrangements.
4. **Heap_decrease_key**(int p, int k, int heap[]): Decreases the value at the p-th position by an amount of k and makes necessary changes.
5. **Heap_increase_key**(int key, int amount, int heap[): Increases the value at the p-th position by an amount of k and makes necessary changes.

Input

First line of input will contain a set of numbers. Show the corresponding min-heap for that.

After that, the input will be like 'function_id necessary_params (if any)'. After each function call, show the output and 'state of the heap'.

Input	**Output**
70 90 60 190 80 170 110 20 50 30 130 -1	Min Heap: 20 30 60 50 70 170 110 190 90 80 130
1	20 20 30 60 50 70 170 110 190 90 80 130
2	20 30 50 60 90 70 170 110 190 130 80
1	30 30 50 60 90 70 170 110 190 130 80
3 45	30 45 60 90 50 170 110 190 130 80 70
4 4 65	25 30 60 45 50 170 110 190 130 80 70
2	25 30 45 60 70 50 170 110 190 130 80
5 1 170	45 50 60 70 80 170 110 190 130 200
3 47	45 47 60 70 50 170 110 190 130 200 80

Note:

- Assume that, we are using 1-based indexing.

(Self-study)

C++ has built-in functions for performing operations on Queue, Heap/ Priority Queue. Check the following links for a better understanding:

Basic STL functions to use queues:

https://www.geeksforgeeks.org/queue-cpp-stl/

https://www.geeksforgeeks.org/heap-using-stl-c/

STL function to swap two queues:
https://www.geeksforgeeks.org/queue-swap-cpp-stl/
https://www.geeksforgeeks.org/heap-using-stl-c/

Task 3:

Mark loves cookies. He wants the sweetness of all his cookies to be greater than the value of **K**. To do this, Mark repeatedly mixes two cookies with the least sweetness. He creates a special combined cookie with:
Sweetness = (1 x Least sweet cookie + 2 x *2nd least sweet cookie*).
He repeats this procedure until all the cookies in his collection have a sweetness ≥ **K**
You are given Mark's cookies. Print the number of operations required to give the cookies a sweetness ≥ **K** Print **-1** if this isn't possible.

Input format
The first line consists of integers **N** representing the number of cookies, and **k**- the minimum required sweetness, separated by a space.
The next line contains **N** integers describing the array ***A*** where A_i is the sweetness of the ith cookie in Mark's collection.

Output format
Output the number of operations that are needed to increase the cookie's sweetness ≥ **K**
Output **-1** if this isn't possible.

Sample Input	**Sample Output**
6 7 12 9 1 3 10 2	2

Explanation
Combine the first two cookies to create a cookie with *sweetness* = 1 x 1 + 2 x 2 = 5
After this operation, the cookies are (3, 5, 9, 10, 12)
Then, combine cookies with sweetness and sweetness, to create a cookie with resulting *sweetness* = 1 x 3 + 2 x 5 = 13
Now, the cookies are (9, 10, 12, 13).
All the cookies have a sweetness >= 7

Thus, **2** operations are required to increase the sweetness.

Note: You should use ***Heap*** to solve this problem. Sorting might be another way of solving this problem, but that will take $O(nlogn)$ in the worst case. But Heap can lead us to a linear solution.

Task 5

Given a set of integers and a value k, return the k most frequent elements.
The first line of input contains the set of integers and the following line contains the value of k. Take input until getting -1.

You may return the answer in any order.

Input	**Output**
10 242 23 12 10 12 23 56 10 10 12 23 56 13 18 14 65 34 242 23 12 10 12 23 56 10 242 23 12 10 12 23 56 13 18 14 65 34 -1 5	10 23 12 56 242
3 1 4 4 5 2 6 1 -1 2	4 1 (1 4 is also valid ans)
1 1 1 2 2 3 -1 2	1 2
7 10 11 5 2 5 5 7 11 8 9 -1 4	5 11 7 10
1 2 2 3 3 -1 2	2 3
1 -1 1	1

Note:

- Assume that the numbers are <= 1000
- Assume that the solution is unique
- You must use **priority queue.**

Task 6:

Given a string s, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string.

Return the sorted string. If there are multiple answers, return any of them.

Input	**Output**	**Explanation**
tree	eert	'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer
cccaaa	aaaccc	Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.
Aabb	bbAa	"bbaA" is also a valid answer, but "Aabb" is incorrect. Note that 'A' and 'a' are treated as two different characters.
acabcdcee	cccaaeedb	
ddabbbc	bbbddac	‘bbbddca’ also valid

Note:

- Utilize the concepts of **priority queue** to solve this task.
- There can be only uppercase/lowercase letters.

Task 8

Given a characters array of `tasks`, representing the tasks a CPU needs to do, where each letter represents a different task. Tasks could be done in any order. Each task is done in one unit of time. For each unit of time, the CPU could complete either one task or just be idle.

However, there is a non-negative integer `n` that represents the cooldown period between two same tasks (the same letter in the array), that is that there must be at least `n` units of time between any two same tasks.

Return *the least number of units of times that the CPU will take to finish all the given tasks*.

Input	**Output**	**Explanation**
A A A B B B 2	8	A -> B -> idle -> A -> B -> idle -> A -> B There is at least 2 units of time between any two same tasks.
A A A B B B 0	6	In this case any permutation of size 6 would work since n = 0. ["A","A","A","B","B","B"] ["A","B","A","B","A","B"] ["B","B","B","A","A","A"] ... And so on.
A A A A A A A B C D E F G 2	16	One possible solution is A -> B -> C -> A -> D -> E -> A -> F -> G -> A -> idle -> idle -> A -> idle -> idle -> A

Task 11

You are given an array of integers ‘stones’ where ‘storen[i]’ is the weight of the i-th stone.
We are playing a game with the stones. On each turn, we choose the **heaviest two stones** and smash them together. Suppose the heaviest two stones have weights x and y, with x<=y. The result of the smash is:

- If x==y, both stones are destroyed.
- If x!=y, the stone of weight x is destroyed, and the stone of weight y has a new weight (y-x).

At the end of the game, there is **at most one stone left**. Return the weight of the last remaining stone. If there are no stones left, return 0.

Input	**Output**	**Explanation**
2 7 4 1 8 1 -1	1	Combine 7,8. State: (2 4 1 1 1) Combine 2,4. State: (2 1 1 1) Combine 2,1. State: (1 1 1) Combine 1,1. State: (1) That's the value of the last stone.
10 10 10 10 10 -1	10	
10 10 5 10 10 10 -1	5	
50 30 10 40 20 -1	10	
50 30 10 40 60 20 -1	10	
10 50 30 10 40 60 20 -1	0	
1 7 5 4 2 2 1 4 8 1 -1	1	
1 7 5 4 2 2 1 4 8 -1	0	
3 3 -1	0	
1 -1	1	